

Debugging Apache Spark Application Issues

Sure! Let's dive into the concept of **user code** in Apache Spark applications.

User code refers to the specific instructions and functions that you write to process data using Spark. Think of it like a recipe you create to bake a cake. The driver program is like the head chef who organizes everything, while the executors are the kitchen staff who carry out the tasks. When you submit your Spark application, the driver creates a SparkContext, which is like setting up your kitchen, and then it sends out tasks to the executors to process the data in parallel, just like different staff members working on various parts of the cake at the same time.

However, just like a recipe can go wrong if you miss an ingredient or use the wrong oven temperature, user code can fail due to errors like syntax mistakes or missing libraries. If something goes wrong, Spark will try to fix it by rerunning the task a few times. If it still fails, it will report the error back to the driver, and the whole application might stop running.

Common Application Issues

Common areas of Spark application issues

- User Code
- Configuration
- Application Dependencies
- Resource Allocation
- Network Communication

- Running a Spark application on a cluster is a complex process with many working parts and many ways in which an application can fail. Common reasons for application failure on a cluster include user code, system and application configurations, application dependencies that are missing or an incorrect version, improper resource allocation, and network communication among cluster nodes.

What is User Code

User code is made up of:

Driver program: code that runs in the driver process

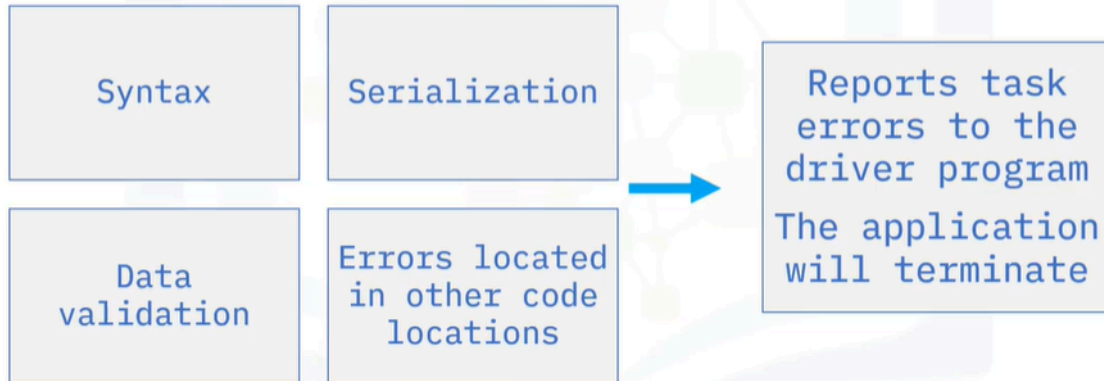
Serialized closures contain the code's necessary functions, classes, and variables

The serialized closures are distributed into the cluster to run in parallel by the executors

- What is user code? User code is made up of the driver program, which runs in the driver process and the functions and variables serialized that the executor runs in parallel. Both the driver and executor processes run the application user code of an application passed to the spark-submit script. The user code in the driver creates the SparkContext and creates jobs based on operations to the DataFrames. These DataFrame operations become serialized closures sent throughout the cluster and run-on executor processes as tasks. The serialized closures contain the necessary functions, classes, and variables needed to run each task.

User Code Issues

- User code can error in the driver



- Spark usually immediately terminates when syntax, serialization, data validation, and other user errors occur. Spark reports task errors to the driver and immediately cancels the related executor tasks, terminating the application. For example, closures are cleaned before being serialized, flushing out most issues right away and ensuring that closures can be serialized and executed remotely.
- User code serialized as a closure might not error until after the user code runs on another executor process. These errors could be due to runtime calculations, network communication issues, or unexpected data issues.
- If a task fails due to an error, Spark can attempt to rerun the task for a set number of retries. If all attempts to run the task fail, Spark reports an error to the driver and the application is terminated. The cause of an application failure can usually be found in the driver event log.

Application dependency issues

Application dependencies include:

- Application files:
 - Source files examples: Python script files, Java JAR files
 - Required data files
- Application libraries

Dependencies must be available for all nodes of the cluster

- A Spark application can have many dependencies including application files such as Python script files, Java JAR files, and even required data files. Applications also depends on the libraries used and their dependencies. Dependencies must be made available on all nodes of the cluster, either by pre-installation or by including the dependencies in the spark-submit script bundled with the application, or as additional arguments. For example, a task will error if a Python library is not installed in the Python environment of the executor process.
- An even more subtle error can occur if a library is installed with different versions on executors that might have different APIs or produce unexpected results. The best way to identify this type of issue is by examining the event log for stack trace errors that identify which libraries the application loaded.

Application Resource Issues

- CPU and memory resources must be available for tasks to run
 - Driver and executor processes require some amount of CPU and memory to run
 - Any Worker with free resources can start processes
 - Resources are acquired until a process completes
 - If resources are not available, Spark retries until a worker is free
 - Lack of resources results in task time-outs, also called task starvation!
-
- Application resources, such as CPU cores and memory, can become an issue if a task is in the scheduling queue and the available workers do not have enough available resources to run the tasks. As a worker finishes a task, the CPU and memory in use are freed, allowing the scheduling of another task. However, if the application asks for more resources that can ever become available, the tasks might never be run and eventually time out. Similarly, suppose that the executors are running long tasks that never finish. In that case, their resources never become available, which also causes future tasks never to run, resulting in a timeout error. You can readily access these errors when you view the UI or event logs.

Examining the Log Files

- Application logs are found in `'work/'` directory named as `work/<application-id>/<stdout|error>`
- Spark standalone writes master/worker logs to the `'log/'` directory

- While the Application UI provides a great deal of information, viewing the log files provides details that give more insights into possible application failure causes. You'll find the application log files in the Spark installation directory, under `work/<application-id>/`, where you will find one log file for `stdout` and one log file for `stderr` output. The application-id is a unique ID that Spark assigns to each application. These log files appear for each executor and driver process that the application runs. Additionally, if you are running a Spark standalone cluster, the master and workers both output log files to the `log/` directory under the Spark installation directory from where they run.

Summary

In this video, you learned that:

- Common reasons for application failure on a cluster include user code, system and application configurations, missing dependencies, improper resource allocation, and network communications
- Application log files, located in the Spark installation directory, will often show the complete details of a failure